Databases

`Databases` in `msfconsole` are used to keep track of your results. It is no mystery that during even more complex machine assessments, much less entire networks, things can get a little fuzzy and complicated due to the sheer amount of search results, entry points, detected issues, discovered credentials, etc.

This is where Databases come into play. `Msfconsole` has built-in support for the PostgreSQL database system. With it, we have direct, quick, and easy access to scan results with the added ability to import and export results in conjunction with third-party tools. Database entries can also be used to configure Exploit module parameters with the already existing findings directly.

Setting up the Database

First, we must ensure that the PostgreSQL server is up and running on our host machine. To do so, input the following command:

PostgreSQL Status

Databases

```
dbcyph0n@htb[/htb]$ sudo service postgresql status

● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/lib/systemd/system/postgresql.service; disabled; vendor preset: disabled)
     Active: active (exited) since Fri 2022-05-06 14:51:30 BST; 3min 51s ago
    Process: 2147 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 2147 (code=exited, status=0/SUCCESS)
        CPU: 1ms

May 06 14:51:30 pwnbox-base systemd[1]: Starting PostgreSQL RDBMS...
May 06 14:51:30 pwnbox-base systemd[1]: Finished PostgreSQL RDBMS.
```

Start PostgreSQL

Databases

```
dbcyph0n@htb[/htb]$ sudo systemctl start postgresql
```

After starting PostgreSQL, we need to create and initialize the MSF database with `msfdb init`.

MSF - Initiate a Database

Databases

```
dbcyph0n@htb[/htb]$ sudo msfdb init

[i] Database already started
[+] Creating database user 'msf'
[+] Creating databases 'msf'
[+] Creating databases 'msf_test'
[+] Creating configuration file '/usr/share/metasploit-framework/config/database.yml'
[+] Creating initial database schema
rake aborted!
NoMethodError: undefined method `without' for #<Bundler::Settings:0x000055dddcf8cba8>
Did you mean? with_options

<SNIP>
```

Sometimes an error can occur if Metasploit is not up to date. This difference that causes the error can happen for several reasons. First, often it helps to update Metasploit again (`apt update`) to solve this problem. Then we can try to reinitialize the MSF database.

Databases

```
dbcyph0n@htb[/htb]$ sudo msfdb init

[i] Database already started
[i] The database appears to be already configured, skipping initialization
```

If the initialization is skipped and Metasploit tells us that the database is already configured, we can recheck the status of the database.

Databases

```
dbcyph0n@htb[/htb]$ sudo msfdb status

● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/lib/systemd/system/postgresql.service; disabled; vendor preset: disabled)
     Active: active (exited) since Mon 2022-05-09 15:19:57 BST; 35min ago
    Process: 2476 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 2476 (code=exited, status=0/SUCCESS)
        CPU: 1ms

May 09 15:19:57 pwnbox-base systemd[1]: Starting PostgreSQL RDBMS...
May 09 15:19:57 pwnbox-base systemd[1]: Finished PostgreSQL RDBMS.

COMMAND   PID     USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
postgres 2458 postgres    5u  IPv6  34336      0t0  TCP localhost:5432 (LISTEN)
postgres 2458 postgres    6u  IPv4  34337      0t0  TCP localhost:5432 (LISTEN)

UID          PID    PPID  C STIME TTY      STAT   TIME CMD
postgres    2458       1  0 15:19 ?        Ss     0:00 /usr/lib/postgresql/13/bin/postgres -D /var/lib/postgres

[+] Detected configuration file (/usr/share/metasploit-framework/config/database.yml)
```

If this error does not appear, which often happens after a fresh installation of Metasploit, then we will see the following when initializing the database:

Databases

```
dbcyph0n@htb[/htb]$ sudo msfdb init

[+] Starting database
[+] Creating database user 'msf'
[+] Creating databases 'msf'
[+] Creating databases 'msf_test'
[+] Creating configuration file '/usr/share/metasploit-framework/config/database.yml'
[+] Creating initial database schema
```

After the database has been initialized, we can start `msfconsole` and connect to the created database simultaneously.

MSF - Connect to the Initiated Database

Databases

```
dbcyph0n@htb[/htb]$ sudo msfdb run

[i] Database already started
```

```
                                .                                   .
 .

      dBBBBBBb  dBBBP dBBBBBBP dBBBBBb  .                        o
       '   dB'                    BBP
    dB'dB'dB' dBBP     dBP     dBP BB
   dB'dB'dB' dBP      dBP     dBP  BB
  dB'dB'dB' dBBBBP   dBP     dBBBBBBB

                                  dBBBBBP  dBBBBBb  dBP    dBBBBP dBP dBBBBBBP
          .                 .                 dB' dBP    dB'.BP
                            |       dBP    dBBBB' dBP    dB'.BP dBP    dBP
                         --o--     dBP    dBP    dBP    dB'.BP dBP    dBP
                            |     dBBBBP dBP    dBBBBP dBBBBP dBP    dBP


                                                                  .
              .
        o                  To boldly go where no
                            shell has gone before


       =[ metasploit v6.1.39-dev                          ]
+ -- --=[ 2214 exploits - 1171 auxiliary - 396 post       ]
+ -- --=[ 616 payloads - 45 encoders - 11 nops            ]
+ -- --=[ 9 evasion                                       ]

msf6>
```

If, however, we already have the database configured and are not able to change the password to the MSF username, proceed with these commands:

MSF - Reinitiate the Database

Databases

```
dbcyph0n@htb[/htb]$ msfdb reinit
dbcyph0n@htb[/htb]$ cp /usr/share/metasploit-framework/config/database.yml ~/.msf4/
dbcyph0n@htb[/htb]$ sudo service postgresql restart
dbcyph0n@htb[/htb]$ msfconsole -q

msf6 > db_status

[*] Connected to msf. Connection type: PostgreSQL.
```

Now, we are good to go. The `msfconsole` also offers integrated help for the database. This gives us a good overview of interacting with and using the database.

MSF - Database Options

Databases

```
msf6 > help database

Database Backend Commands
=========================

    Command           Description
    -------           -----------
    db_connect        Connect to an existing database
    db_disconnect     Disconnect from the current database instance
    db_export         Export a file containing the contents of the database
    db_import         Import a scan result file (filetype will be auto-detected)
    db_nmap           Executes nmap and records the output automatically
    db_rebuild_cache  Rebuilds the database-stored module cache
```

```
    db_status         Show the current database status
    hosts             List all hosts in the database
    loot              List all loot in the database
    notes             List all notes in the database
    services          List all services in the database
    vulns             List all vulnerabilities in the database
    workspace         Switch between database workspaces


msf6 > db_status

[*] Connected to msf. Connection type: postgresql.
```

Using the Database

With the help of the database, we can manage many different categories and hosts that we have analyzed. Alternatively, the information about them that we have interacted with using Metasploit. These databases can be exported and imported. This is especially useful when we have extensive lists of hosts, loot, notes, and stored vulnerabilities for these hosts. After confirming that the database is successfully connected, we can organize our `Workspaces`.

Workspaces

We can think of `Workspaces` the same way we would think of folders in a project. We can segregate the different scan results, hosts, and extracted information by IP, subnet, network, or domain.

To view the current Workspace list, use the `workspace` command. Adding a `-a` or `-d` switch after the command, followed by the workspace's name, will either `add` or `delete` that workspace to the database.

Databases

```
msf6 > workspace

* default
```

Notice that the default Workspace is named `default` and is currently in use according to the `*` symbol. Type the `workspace [name]` command to switch the presently used workspace. Looking back at our example, let us create a workspace for this assessment and select it.

Databases

```
msf6 > workspace -a Target_1

[*] Added workspace: Target_1
[*] Workspace: Target_1


msf6 > workspace Target_1

[*] Workspace: Target_1


msf6 > workspace

  default
* Target_1
```

To see what else we can do with Workspaces, we can use the `workspace -h` command for the help menu related to Workspaces.

Databases

Databases

```
msf6 > workspace -h

Usage:
    workspace                 List workspaces
    workspace -v              List workspaces verbosely
    workspace [name]          Switch workspace
    workspace -a [name] ...   Add workspace(s)
    workspace -d [name] ...   Delete workspace(s)
    workspace -D              Delete all workspaces
    workspace -r     Rename workspace
    workspace -h              Show this help information
```

Importing Scan Results

Next, let us assume we want to import a `Nmap scan` of a host into our Database's Workspace to understand the target better. We can use the `db_import` command for this. After the import is complete, we can check the presence of the host's information in our database by using the `hosts` and `services` commands. Note that the `.xml` file type is preferred for `db_import`.

Stored Nmap Scan

Databases

```
dbcyph0n@htb[/htb]$ cat Target.nmap

Starting Nmap 7.80 ( https://nmap.org ) at 2020-08-17 20:54 UTC
Nmap scan report for 10.10.10.40
Host is up (0.017s latency).
Not shown: 991 closed ports
PORT      STATE SERVICE      VERSION
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
49152/tcp open  msrpc        Microsoft Windows RPC
49153/tcp open  msrpc        Microsoft Windows RPC
49154/tcp open  msrpc        Microsoft Windows RPC
49155/tcp open  msrpc        Microsoft Windows RPC
49156/tcp open  msrpc        Microsoft Windows RPC
49157/tcp open  msrpc        Microsoft Windows RPC
Service Info: Host: HARIS-PC; OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 60.81 seconds
```

Importing Scan Results

Databases

```
msf6 > db_import Target.xml

[*] Importing 'Nmap XML' data
[*] Import: Parsing with 'Nokogiri v1.10.9'
[*] Importing host 10.10.10.40
[*] Successfully imported ~/Target.xml


msf6 > hosts

Hosts
=====

address      mac  name  os_name  os_flavor  os_sp  purpose  info  comments
-------      ---  ----  -------  ---------  -----  -------  ----  --------
10.10.10.40             Unknown                    device
```

```
msf6 > services

Services
========

host         port   proto  name          state  info
----         ----   -----  ----          -----  ----
10.10.10.40  135    tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  139    tcp    netbios-ssn   open   Microsoft Windows netbios-ssn
10.10.10.40  445    tcp    microsoft-ds  open   Microsoft Windows 7 - 10 microsoft-ds workgroup: WORKGROUP
10.10.10.40  49152  tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  49153  tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  49154  tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  49155  tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  49156  tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  49157  tcp    msrpc         open   Microsoft Windows RPC
```

Using Nmap Inside MSFconsole

Alternatively, we can use Nmap straight from msfconsole! To scan directly from the console without having to background or exit the process, use the `db_nmap` command.

MSF - Nmap

Databases

```
msf6 > db_nmap -sV -sS 10.10.10.8

[*] Nmap: Starting Nmap 7.80 ( https://nmap.org ) at 2020-08-17 21:04 UTC
[*] Nmap: Nmap scan report for 10.10.10.8
[*] Nmap: Host is up (0.016s latency).
[*] Nmap: Not shown: 999 filtered ports
[*] Nmap: PORT   STATE SERVICE VERSION
[*] Nmap: 80/TCP open  http    HttpFileServer httpd 2.3
[*] Nmap: Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
[*] Nmap: Service detection performed. Please report any incorrect results at https://nmap.org/submit/
[*] Nmap: Nmap done: 1 IP address (1 host up) scanned in 11.12 seconds


msf6 > hosts

Hosts
=====

address      mac  name  os_name  os_flavor  os_sp  purpose  info  comments
-------      ---  ----  -------  ---------  -----  -------  ----  --------
10.10.10.8              Unknown                    device
10.10.10.40             Unknown                    device


msf6 > services

Services
========

host         port   proto  name          state  info
----         ----   -----  ----          -----  ----
10.10.10.8   80     tcp    http          open   HttpFileServer httpd 2.3
10.10.10.40  135    tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  139    tcp    netbios-ssn   open   Microsoft Windows netbios-ssn
10.10.10.40  445    tcp    microsoft-ds  open   Microsoft Windows 7 - 10 microsoft-ds workgroup: WORKGROUP
10.10.10.40  49152  tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  49153  tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  49154  tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  49155  tcp    msrpc         open   Microsoft Windows RPC
10.10.10.40  49156  tcp    msrpc         open   Microsoft Windows RPC
```

```
10.10.10.40   49157  tcp    msrpc          open   Microsoft Windows RPC
```

Data Backup

After finishing the session, make sure to back up our data if anything happens with the PostgreSQL service. To do so, use the `db_export` command.

MSF - DB Export

Databases

```
msf6 > db_export -h

Usage:
    db_export -f <format> [filename]
    Format can be one of: xml, pwdump
[-] No output file was specified


msf6 > db_export -f xml backup.xml

[*] Starting export of workspace default to backup.xml [ xml ]...
[*] Finished export of workspace default to backup.xml [ xml ]...
```

This data can be imported back to msfconsole later when needed. Other commands related to data retention are the extended use of `hosts`, `services`, and the `creds` and `loot` commands.

Hosts

The `hosts` command displays a database table automatically populated with the host addresses, hostnames, and other information we find about these during our scans and interactions. For example, suppose `msfconsole` is linked with scanner plugins that can perform service and OS detection. In that case, this information should automatically appear in the table once the scans are completed through msfconsole. Again, tools like Nessus, NexPose, or Nmap will help us in these cases.

Hosts can also be manually added as separate entries in this table. After adding our custom hosts, we can also organize the format and structure of the table, add comments, change existing information, and more.

MSF - Stored Hosts

Databases

```
msf6 > hosts -h

Usage: hosts [ options ] [addr1 addr2 ...]

OPTIONS:
  -a,--add          Add the hosts instead of searching
  -d,--delete       Delete the hosts instead of searching
  -c <col1,col2>    Only show the given columns (see list below)
  -C <col1,col2>    Only show the given columns until the next restart (see list below)
  -h,--help         Show this help information
  -u,--up           Only show hosts which are up
  -o <file>         Send output to a file in CSV format
  -O <column>       Order rows by specified column number
  -R,--rhosts       Set RHOSTS from the results of the search
  -S,--search       Search string to filter by
  -i,--info         Change the info of a host
  -n,--name         Change the name of a host
  -m,--comment      Change the comment of a host
```

```
  -m,--comment      Change the comment of a host
  -t,--tag          Add or specify a tag to a range of hosts

Available columns: address, arch, comm, comments, created_at, cred_count, detected_arch, exploit_attempt_count
```

Services

The `services` command functions the same way as the previous one. It contains a table with descriptions and information on services discovered during scans or interactions. In the same way as the command above, the entries here are highly customizable.

MSF - Stored Services of Hosts

Databases

```
msf6 > services -h

Usage: services [-h] [-u] [-a] [-r <proto>] [-p <port1,port2>] [-s <name1,name2>] [-o <filename>] [addr1 addr2

  -a,--add          Add the services instead of searching
  -d,--delete       Delete the services instead of searching
  -c <col1,col2>    Only show the given columns
  -h,--help         Show this help information
  -s <name>         Name of the service to add
  -p <port>         Search for a list of ports
  -r <protocol>     Protocol type of the service being added [tcp|udp]
  -u,--up           Only show services which are up
  -o <file>         Send output to a file in csv format
  -O <column>       Order rows by specified column number
  -R,--rhosts       Set RHOSTS from the results of the search
  -S,--search       Search string to filter by
  -U,--update       Update data for existing service

Available columns: created_at, info, name, port, proto, state, updated_at
```

Credentials

The `creds` command allows you to visualize the credentials gathered during your interactions with the target host. We can also add credentials manually, match existing credentials with port specifications, add descriptions, etc.

MSF - Stored Credentials

Databases

```
msf6 > creds -h

With no sub-command, list credentials. If an address range is
given, show only credentials with logins on hosts within that
range.

Usage - Listing credentials:
  creds [filter options] [address range]

Usage - Adding credentials:
  creds add uses the following named parameters.
    user      :  Public, usually a username
    password  :  Private, private_type Password.
    ntlm      :  Private, private_type NTLM Hash.
    Postgres  :  Private, private_type Postgres MD5
    ssh-key   :  Private, private_type SSH key, must be a file path.
    hash      :  Private, private_type Nonreplayable hash
    jtr       :  Private, private_type John the Ripper hash type.
    realm     :  Realm,
    realm-type:  Realm, realm type (domain db2db sid nsdb nsyns wildcard), defaults to domain
```

```
    realm-type:  Realm, realm_type (domain db2db sid pgdb rsync wildcard), defaults to domain.

Examples: Adding
   # Add a user, password and realm
   creds add user:admin password:notpassword realm:workgroup
   # Add a user and password
   creds add user:guest password:'guest password'
   # Add a password
   creds add password:'password without username'
   # Add a user with an NTLMHash
   creds add user:admin ntlm:E2FC15074BF7751DD408E6B105741864:A1074A69B1BDE45403AB680504BBDD1A
   # Add a NTLMHash
   creds add ntlm:E2FC15074BF7751DD408E6B105741864:A1074A69B1BDE45403AB680504BBDD1A
   # Add a Postgres MD5
   creds add user:postgres postgres:md5be86a79bf2043622d58d5453c47d4860
   # Add a user with an SSH key
   creds add user:sshadmin ssh-key:/path/to/id_rsa
   # Add a user and a NonReplayableHash
   creds add user:other hash:d19c32489b870735b5f587d76b934283 jtr:md5
   # Add a NonReplayableHash
   creds add hash:d19c32489b870735b5f587d76b934283

General options
  -h,--help             Show this help information
  -o <file>             Send output to a file in csv/jtr (john the ripper) format.
                        If the file name ends in '.jtr', that format will be used.
                        If file name ends in '.hcat', the hashcat format will be used.
                        CSV by default.
  -d,--delete           Delete one or more credentials

Filter options for listing
  -P,--password <text>  List passwords that match this text
  -p,--port <portspec>  List creds with logins on services matching this port spec
  -s <svc names>        List creds matching comma-separated service names
  -u,--user <text>      List users that match this text
  -t,--type <type>      List creds that match the following types: password,ntlm,hash
  -O,--origins <IP>     List creds that match these origins
  -R,--rhosts           Set RHOSTS from the results of the search
  -v,--verbose          Don't truncate long password hashes

Examples, John the Ripper hash types:
  Operating Systems (starts with)
    Blowfish ($2a$)   : bf
    BSDi     (_)      : bsdi
    DES               : des,crypt
    MD5      ($1$)    : md5
    SHA256   ($5$)    : sha256,crypt
    SHA512   ($6$)    : sha512,crypt
  Databases
    MSSQL             : mssql
    MSSQL 2005        : mssql05
    MSSQL 2012/2014   : mssql12
    MySQL < 4.1       : mysql
    MySQL >= 4.1      : mysql-sha1
    Oracle            : des,oracle
    Oracle 11         : raw-sha1,oracle11
    Oracle 11 (H type): dynamic_1506
    Oracle 12c        : oracle12c
    Postgres          : postgres,raw-md5

Examples, listing:
  creds               # Default, returns all credentials
  creds 1.2.3.4/24    # Return credentials with logins in this range
  creds -O 1.2.3.4/24 # Return credentials with origins in this range
  creds -p 22-25,445  # nmap port specification
  creds -s ssh,smb    # All creds associated with a login on SSH or SMB services
  creds -t NTLM       # All NTLM creds
  creds -j md5        # All John the Ripper hash type MD5 creds

Example, deleting:
  # Delete all SMB credentials
  creds -d -s smb
```

Loot

The `loot` command works in conjunction with the command above to offer you an at-a-glance list of owned services and users. The loot, in this case, refers to hash dumps from different system types, namely hashes, passwd, shadow, and more.

MSF - Stored Loot

Databases

```
msf6 > loot -h

Usage: loot [options]
 Info: loot [-h] [addr1 addr2 ...] [-t <type1,type2>]
  Add: loot -f [fname] -i [info] -a [addr1 addr2 ...] -t [type]
  Del: loot -d [addr1 addr2 ...]

  -a,--add          Add loot to the list of addresses, instead of listing
  -d,--delete       Delete *all* loot matching host and type
  -f,--file         File with contents of the loot to add
  -i,--info         Info of the loot to add
  -t <type1,type2>  Search for a list of types
  -h,--help         Show this help information
  -S,--search       Search string to filter by
```

Previous Next

Mark Complete & Next

Cheat Sheet

Table of Contents

Introduction

Preface

Introduction to Metasploit

Introduction to MSFconsole

MSF Components

Modules

Targets

Payloads

Encoders

Databases

Plugins & Mixins

MSF Sessions

Sessions & Jobs

Meterpreter

Additional Features

Writing & Importing Modules

Introduction to MSFVenom

Firewall and IDS/IPS evasion

Metasploit-Framework Updates - August 2020

My Workstation

OFFLINE

Start Instance

∞ / 1 spawns left